SYNEXUS SOFTWARE TECHNOLOGIES
Full Stack Development Track — Phase 1 Internship Evaluation

Week 3 Submission

Server-Side Filtering, Pagination & Dynamic UI

Track	Duration	Format	Stack
Full Stack	Week 3 of 4	Practical Build	React + Express + MongoDB

Task Summary

This submission extends the Week 1/2 Inventory Management module with a highly interactive data table: server-side search, category filtering, sorting, and pagination, with the current view reflected directly in the URL. The API only ever returns the exact slice of data requested, rather than fetching the full collection and filtering in the browser.

Repository

github.com/Rahmatsoga/week1-task → Week3_Task/

1. What Was Implemented

- Extended the Item schema with subCategory and an embedded variants array (richer, nested records — e.g. product size variants), plus database indexes on name, category, price, and createdAt.
- GET /api/items now accepts search, category, sortBy, order, page, and limit query parameters, resolved as a single efficient MongoDB query rather than fetching everything and filtering in JavaScript.
- Added GET /api/items/categories, returning the distinct categories actually present in the database to power the filter dropdown from real data.
- Debounced search input (400ms) — the API is only called once the user pauses typing, not on every keystroke.
- All search/filter/sort/page state lives in the URL via useSearchParams, so refreshing, sharing a link, or using the browser's back/forward buttons all restore the exact same view.
- A 26-item seed script spread across 5 categories, with several items given real variants, so pagination and filtering are meaningful to test.

Evaluation Criteria Coverage

Criterion	Marks	Status
Backend Query Logic	6	Complete
Pagination	5	Complete
Search & Filters	5	Complete
URL State	4	Complete
Performance	5	Complete

2. How It Was Implemented

2.1 Server-Side Query Building

GET /api/items dynamically builds a MongoDB filter object from the incoming query parameters: a case-insensitive regex match on name for search, an exact match on category, and a sort object built from an allowlist of sortable fields (name, price, quantity, createdAt) to prevent an arbitrary or unindexed field being requested. Pagination uses `.skip()` and `.limit()`, with the item count and the page of results fetched in parallel via `Promise.all`.

2.2 Why Server-Side Rather Than Client-Side Filtering

An earlier, simpler approach would fetch every item once and filter/sort/paginate in the browser with JavaScript. This was deliberately avoided: it means the server does unnecessary work sending data that's immediately discarded, and performance degrades as the dataset grows into the thousands. Instead, MongoDB performs the filtering, sorting, and slicing itself, backed by indexes on the fields being queried, so only the small slice actually needed is ever sent over the network.

2.3 Debounced Search

A hand-written `useDebounce` hook (no external dependency) delays updating the search value until the user has paused typing for 400ms. Without this, a 14-character search term could fire 14 separate API requests; with it, only one request fires per pause in typing, avoiding both wasted requests and the risk of an older, slower response overwriting a newer one.

2.4 URL as the Source of Truth

Search, category, sort, and page values are read from and written to the URL via react-router-dom's `useSearchParams`, rather than being held only in local component state. This means a refreshed page, a shared link, or the browser's back/forward buttons all correctly reproduce the exact same filtered, sorted, paginated view — verified directly in this week's testing (see Section 4).

2.5 Richer Data Model

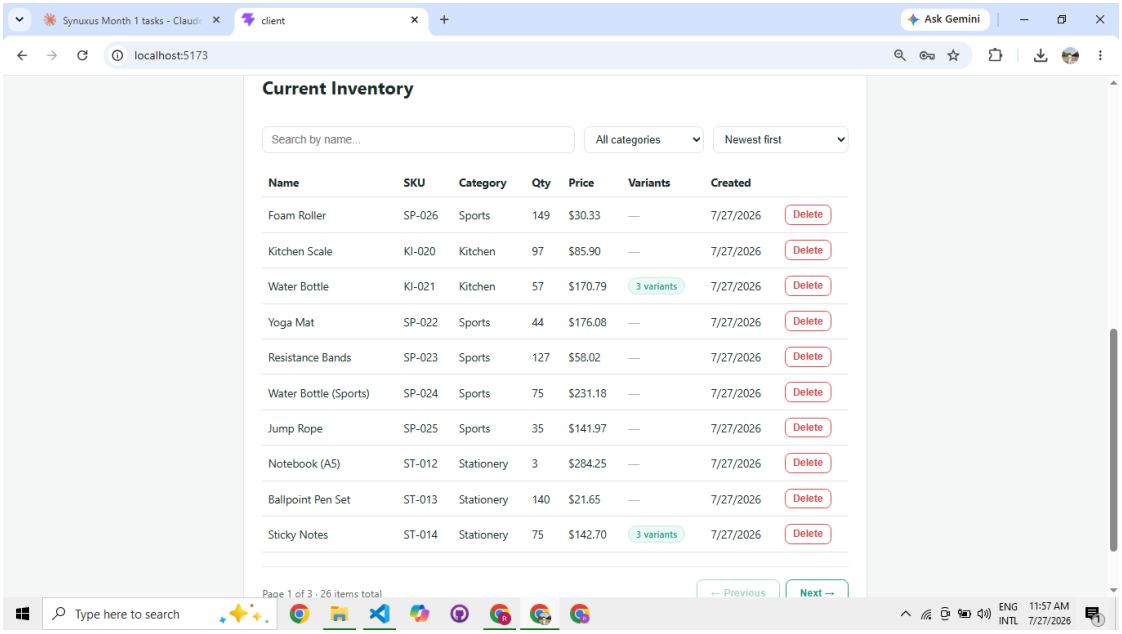
The Item schema now includes an embedded `variants` array (sub-documents with a label, SKU, and stock count), satisfying the "products with variants" requirement. A `subCategory` field was also added to support nested category paths going forward.

3. Demo Walkthrough & Evidence

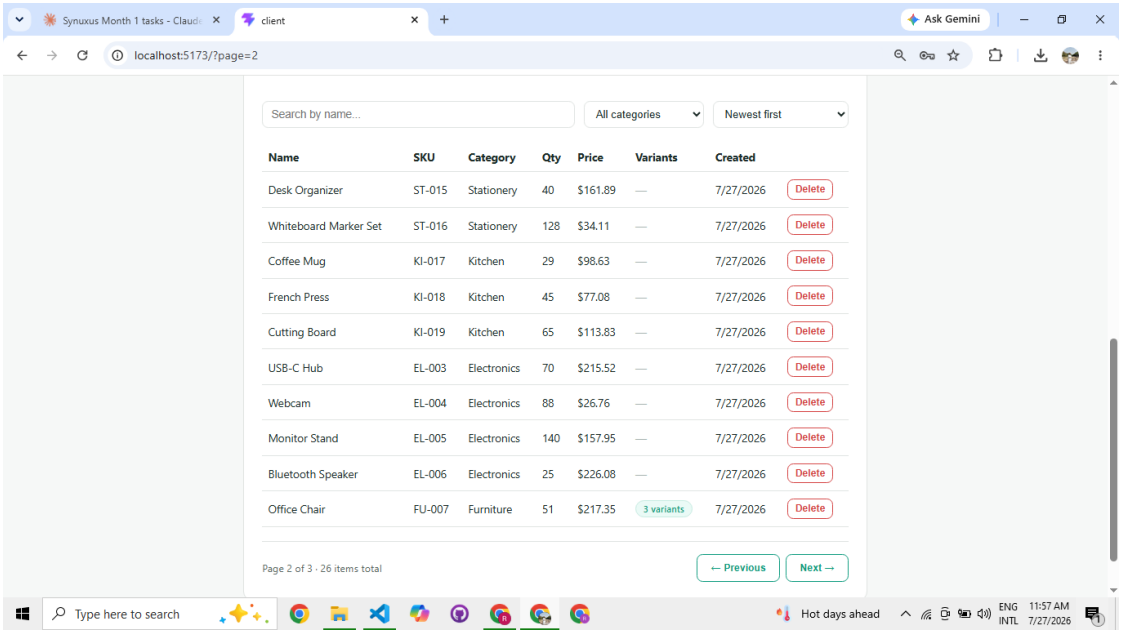
The steps below verify the reviewer checkpoint for Week 3: *"Reviewer can change filters/search/page and see URL and server results update correctly."*

3.1 Pagination across pages

26 seeded items are correctly split across 3 pages of 10. The URL's `page` parameter updates as Next/Previous are clicked, and controls disable appropriately at the first/last page:

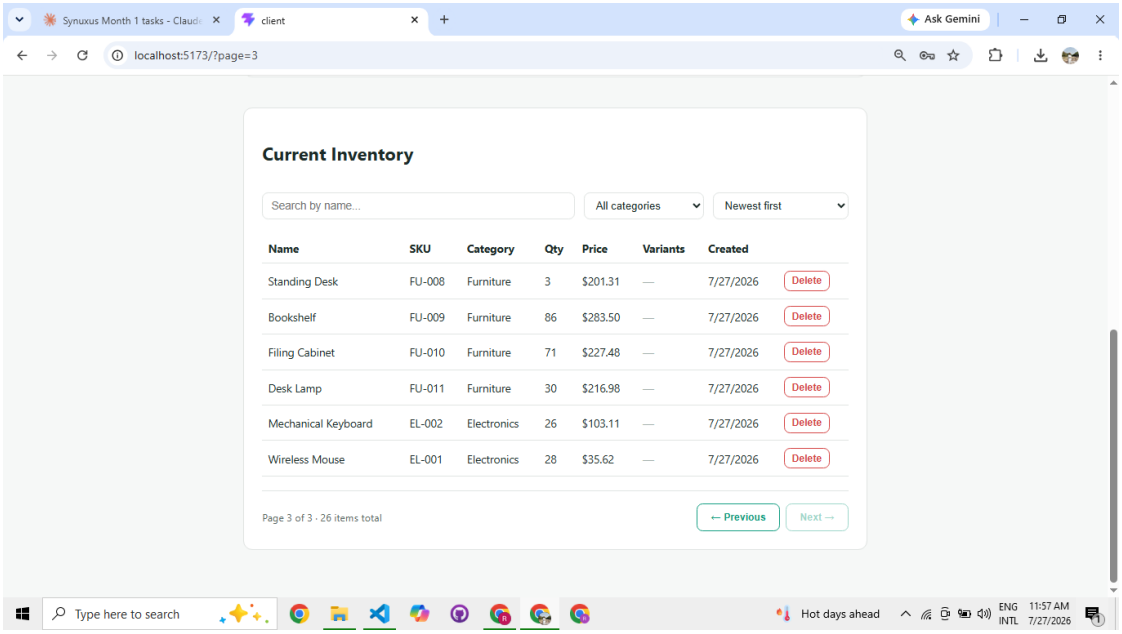


Page 1 of 3 — Next enabled, Previous disabled.



Page 2 of 3, reflected in the URL as ?page=2. A variant badge is visible on "Office Chair."

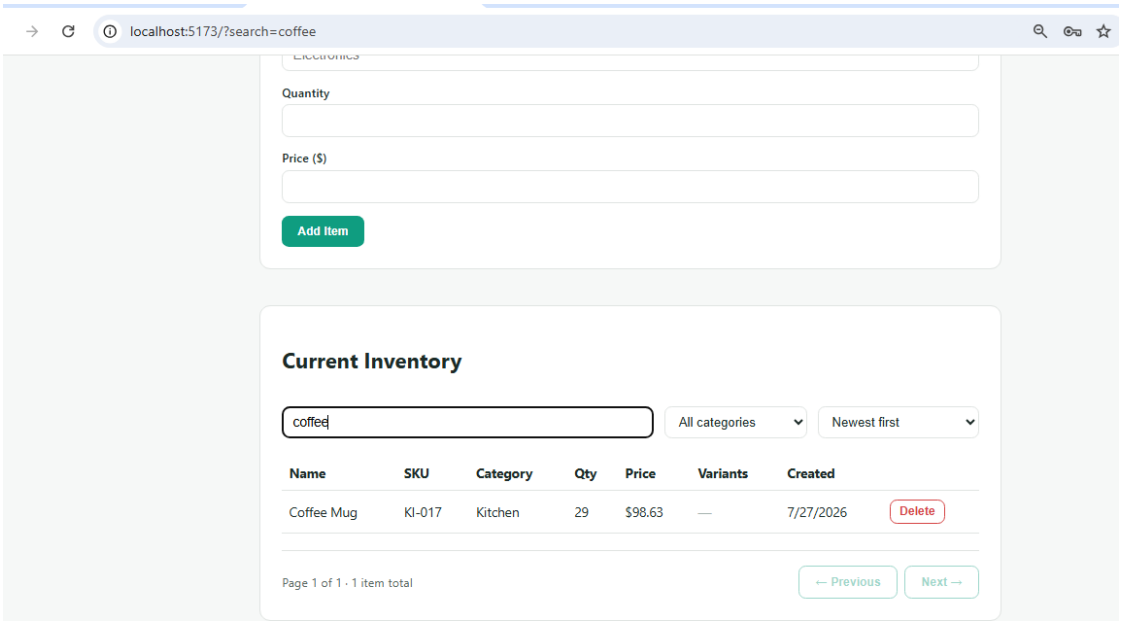
3.2 Final page — pagination bounds



Page 3 of 3 — Next correctly disabled at the last page.

3.3 Search filtering

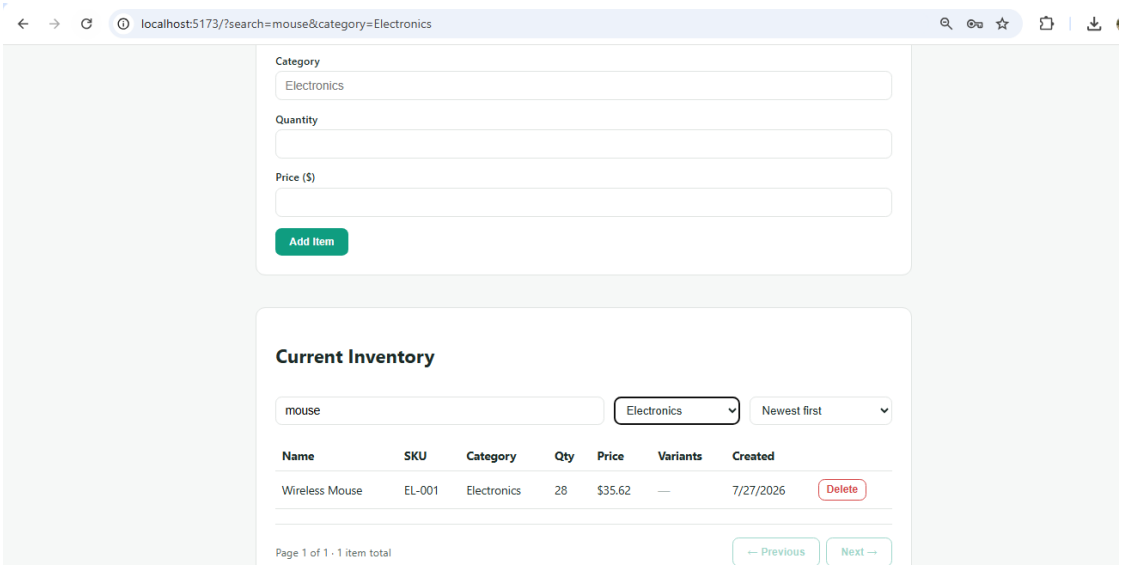
Searching "coffee" correctly narrows the full 26-item collection down to the single matching item, with the URL updated to `?search=coffee`:



Search for "coffee" correctly returns only "Coffee Mug."

3.4 Search combined with category filter

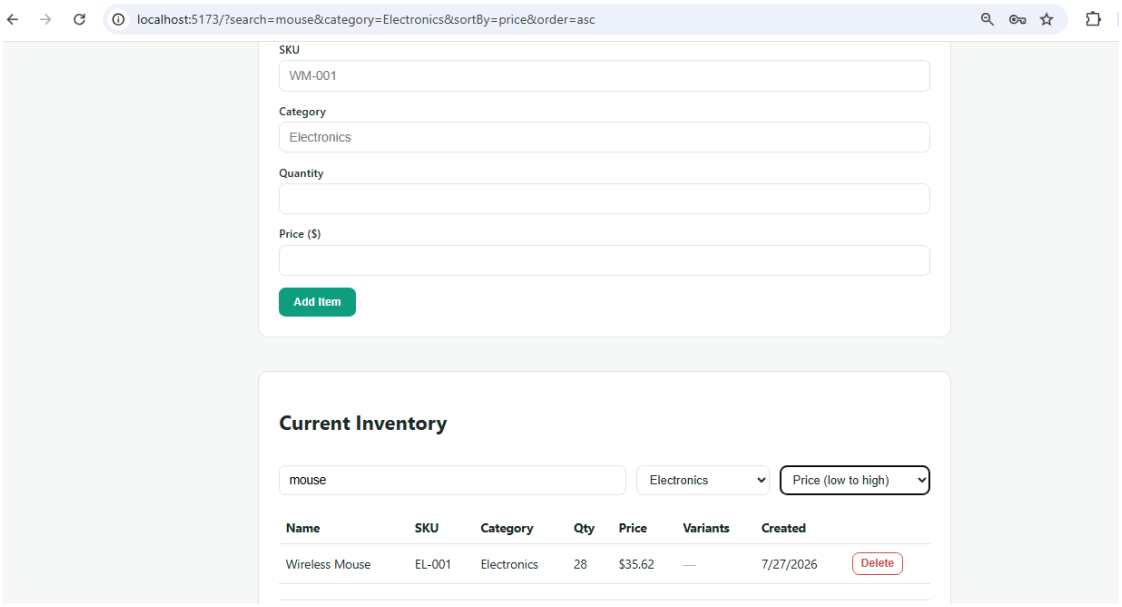
Search and category filter apply together rather than overwriting each other, with both reflected in the URL simultaneously:



Search "mouse" + category "Electronics" combined: ?search=mouse&category;=Electronics.

3.5 Search, filter, and sort combined

Adding a sort option on top of an active search and filter correctly stacks all three in a single URL, confirming the query-building logic composes correctly rather than handling each parameter in isolation:



All three combined: ?search=mouse&category;=Electronics&sortBy;=price■=asc.

3.6 Verified behavior

- Pagination correctly splits 26 items across 3 pages, with Previous/Next enabling and disabling appropriately at the boundaries.
- The URL's page parameter updates on navigation and is restored correctly on refresh.
- Search narrows results correctly and updates the URL's search parameter.
- Category filtering narrows results correctly and updates the URL's category parameter.
- Sorting changes result order correctly and updates the URL's sortBy/order parameters.
- Search, category, and sort all combine correctly in a single query rather than overwriting one another.
- Refreshing the browser (F5) while search/filter/sort/page are active restores the exact same view, confirming the URL — not component state — is the source of truth.

4. Tradeoffs & Known Limitations

- The create-item form doesn't yet expose subCategory or variants for input; the Week 3 focus was the read/query side (filtering, search, pagination) rather than the create form.
- Search matches only the name field; searching across category/SKU as well would be a reasonable next step.
- No debounce/throttle was needed for pagination or the filter dropdowns, since those are discrete click/select events rather than continuous ones — debounce was applied only where it's actually needed: the free-text search input.

5. Improvements With More Time

- Add subCategory and variant management to the create/edit form.
- Add multi-field search (name + SKU + category) using a MongoDB text index.
- Add a compound index (e.g. category + price) if filter+sort combinations on multiple fields become a common access pattern.
- Add an "items per page" selector in the UI (the backend already supports an arbitrary limit up to 100).
